CANN BENCH: BENCHMARKING AGENT GENERATED KERNELS AGAINST REAL NPU AND ALGORITHMIC LIMITS

Xue-Jian Gao[*], Deng Pan[*], Yueming Su[*], Jiasheng Li, Bin Du, Fengming Zhu, Chengdi Ma, Junyi Fan, Qichen Liao, Chengqiu Hu, Xinxian Chen, Lingchao Zheng, Jun Li, Jiwei Yang, Yuwei Fan[#]

Huawei

ABSTRACT

AI agents are now capable of writing, compiling, and iteratively optimizing low-level operator kernels on different hardware platforms. Existing benchmarks, however, focus almost exclusively on CUDA and Triton, leaving hardware ecosystems with less-exposed programming models without a common evaluation baseline. We present **CANN Bench**, an open benchmark for AI-generated operator code on Huawei's Ascend NPU. The current release covers **53 operators** and **1060 test cases** organized into four difficulty tiers—from simple elementwise primitives to MoE dispatch and FlashAttention kernels—spanning FP16, BF16, FP32, and INT8 precision formats. Evaluation adopts a **three-dimensional weighted composite score** that treats compilation, functional correctness, and performance as independent axes, providing a principled reward signal for kernel-generation agents. Performance is graded against an out-of-the-box PyTorch-on-Ascend baseline and an analytical per-case hardware limit on real NPU hardware, ensuring scores reflect genuine optimization headroom rather than measurement artifacts. The evaluation harness is designed to resist reward hacking from the ground up. CANN Bench is versioned within the official CANN repository and is designed for long-term community co-construction, providing the Ascend ecosystem with a quantitative, reproducible, and sustainably maintained yardstick for AI operator-authoring capability.

***K*eywords** Benchmark · Ascend NPU · CANN · Kernel Generation · LLM Agents

1 Introduction

1.1 Why an Ascend-Oriented Benchmark Now

Code-generation agents now write, compile, and profile operator kernels end to end [1–5], and the benchmarks they target supply the execution-feedback signal that drives each next round of optimization [6–9]. On Huawei's Ascend NPU, kernel generation runs through CANN—Huawei's heterogeneous computing architecture that bridges upper-layer AI frameworks and the underlying Ascend processors. Writing a kernel under CANN requires explicit control of the memory hierarchy, tile shapes, and Vector–Cube pipeline scheduling, a programming surface analogous to CUDA's on NVIDIA silicon. Recent benchmarks have begun to include Ascend targets [10–15], but they either treat Ascend as one platform among many or are coupled to a specific agent-training recipe. The Ascend ecosystem therefore still lacks a shared evaluation scale for AI-generated CANN kernels—one independent of any single training recipe and maintained as a cross-project reference.

[*] These authors contributed equally: Xue-Jian Gao, Deng Pan, Yueming Su

[#] To whom the correspondence should be addressed: fanyuwei2@huawei.com

The catalog of operators that must be supported continues to expand. Each new model family introduces new attention variants and MoE routing patterns that have to be re-tiled and re-scheduled for Ascend under CANN. Precision compounds the growth: modern pipelines mix FP16 and BF16 with low-precision formats such as FP4 and MXFP4 [16, 17], each with distinct granularity and performance trade-offs. Workload churn and precision growth together push the operator catalog beyond what hand-authored kernels can track. An Ascend-oriented benchmark therefore needs to evolve with the hardware and software stack rather than remain fixed at release. We define six design criteria for such a benchmark below.

1.2 Six Criteria for an Ascend-Oriented Benchmark

We organize the six criteria into two layers. The first four concern the task set; the remaining two concern benchmark delivery and maintenance.

C1. Difficulty span. The task set must span the full difficulty range, from elementwise and reduction primitives, through compute-bound cores such as Conv and Matmul, up to fusion-level operators such as MoE dispatch and FlashAttention. Without this range, scores cluster at the trivial or fusion-only extremes and fail to separate capability tiers.

C2. Workload provenance. Tasks come from real Ascend production workloads—LLM training, quantized inference, and vision/multimodality pipelines—rather than from synthetic cases whose failure modes do not transfer.

C3. Precision and quantization coverage. FP16 and BF16 are the baseline. Quantized integer paths (INT8 with per-tensor and per-channel granularities) are equally important in production under CANN, and lower-precision floating-point formats such as FP8, HiF8, MXFP8, FP4, and MXFP4 [16, 17] are on the near-term roadmap (§1.6).

C4. Evaluation on three axes. Compilation, functional correctness, and performance on Ascend NPUs all contribute to the final weighted score; failing any one of them strictly limits the credit a submission can receive on the remaining axes.

C5. Self-contained reproducibility kit. Each operator ships with a specification, a golden implementation, public test cases, and a performance baseline, so that third parties can reproduce the evaluation.

C6. Anti-reward-hacking and sustained evolution. The harness must detect reward-hacking attempts such as bypassing accuracy assertions, memorizing outputs, or hard-coding shapes. The task set and evaluation protocol must also evolve with Ascend hardware, CANN releases, and model families rather than remain fixed at release.

1.3 Existing Landscape Against These Criteria

Measured against the criteria in §1.2, existing benchmarks cover only subsets of the design space. We group the closest prior work by target hardware and summarize their positions in Table 1.

The CUDA and Triton ecosystem hosts the largest body of kernel-generation benchmarks. KernelBench [18] decomposes evaluation into compilation, correctness, and speedup across 250 PyTorch-to-CUDA tasks. TritonBench [19] applies a similar three-axis setup to 184 real-world Triton operators, where DSL-concept hallucination is a dominant failure mode. SOL-ExecBench [20] anchors performance to hardware Speed-of-Light bounds on NVIDIA Blackwell and adds a sandboxed harness for timing. All three target NVIDIA platforms. Their task sets do not transfer directly to Ascend: CANN exposes different tiling semantics and a different multi-level buffer structure, so a mechanical port fails C2 (workload provenance).

The same non-transfer appears on other NPU stacks. NPUEval [21] evaluates frontier LLMs on 102 AMD-NPU kernel tasks and reports an average vectorization score of around 10%. KernelCraft [22] studies close-to-metal kernel generation across the PLENA, AMD NPU, and Coral NPU ISAs on 33 tasks. Neither targets CANN nor Ascend.

Two benchmarks target the Ascend ecosystem more directly, but neither fully satisfies our six criteria. MultiKernelBench [10] covers 285 tasks across 14 kernel categories and three hardware platforms—NVIDIA

GPUs, Huawei Ascend NPUs, and Google TPUs—using author-provided references and a category-aware one-shot prompting scheme. Breadth costs per-platform depth: Ascend tasks are one slice of the total, so per-operator specification depth is limited (partial C5) and Ascend precision coverage is narrow (partial C3). NPUKernelBench, released alongside the AscendKernelGen fine-tuning recipe [11], covers Ascend tasks at three difficulty levels; AscendKernelGen reports a Level-2 compile rate rising from 0% to 95.5% Pass@10 with chain-of-thought data plus reinforcement learning from on-device execution feedback. Two limitations weaken its role as a shared reference. First, on the static-shape track that constitutes a substantial portion of NPUKernelBench, all test cases for a given task correspond to a single fixed input configuration [11], so a kernel specialized to one canonical shape already passes per-task evaluation. Second, the benchmark is released and evolved alongside the AscendKernelGen recipe rather than on an independent maintenance track, so its task set tracks the cadence of one training pipeline rather than the broader Ascend ecosystem (partial C6).

Several kernel-generation methods now target CANN directly. AscendCraft [12] applies DSL-guided transcompilation through a constraint-directed intermediate representation. AscendOptimizer [13] runs an episodic evolutionary agent with optimization-rewind over 127 operators from the `cann-ops` repository and reports a 1.19× geometric-mean speedup. EvoKernel [14] addresses cold-start and continual refinement through value-driven memory; on a self-constructed NPU variant of KernelBench it lifts frontier-model correctness from 11% to 83%. KernelCAT [15] reports deployment-side gains on FlashAttention and DeepSeek-OCR-2 on Ascend 910B2. Each of these efforts uses either a cross-platform benchmark, a custom task set, or an author-chosen operator collection, so they still lack a common evaluation scale.

C6 (anti-reward-hacking and sustained evolution) has a concrete precedent. KernelBench has documented silent-dispatch exploits in practice: submissions that call high-level operators such as `cuBLAS` in place of the generated CUDA kernel, or no-op kernels whose output memory is reused from the reference tensor [23–25]. Table 1 positions the above benchmarks along five axes: golden-reference provenance, scoring format, generalization discipline, anti-reward-hacking stance, and maintenance mode. Among publicly described Ascend NPU benchmarks, we are not aware of another that combines co-location in the official CANN repository, maintenance alongside vendor operator contracts, and a dedicated leaderboard protocol; CANN Bench is designed to cover all three.

1.4 CANN Bench: Dataset, Scoring, and Online Leaderboard

CANN Bench is an operator-generation benchmark maintained in the official CANN repository. Its specifications, golden implementations, test cases, and performance baselines sit alongside Huawei's vendor-maintained operator contracts (C2, C5). The initial release targets Ascend 910B2 with a kernel-DSL-agnostic specification format designed to extend to Triton, PyPTO, TileLang, and other CANN-compatible paradigms in future releases.

Dataset. CANN Bench currently contains 53 operators and 1060 test cases, distributed 8/16/21/8 across four difficulty levels (L1–L4) that track Ascend hardware-unit usage, from Vector-only kernels at L1 to fused Matrix–Vector pipelines at L4. §2 lists representative operators, the full per-level breakdown, and the four per-operator reproducibility artifacts (`desc.md`, `proto.yaml`, `cases.csv`, `golden.py`).

Public and hidden case split. Each operator's cases split into a 20-case public set for local iteration and an 80-case hidden set that the harness retains for leaderboard evaluation. The leaderboard aggregates both splits, so the hidden set guards against fit to disclosed inputs alone.

Three-dimensional weighted scoring. Compilation acts at the operator level, while functional correctness and performance are scored case by case; the three signals are combined into a per-operator weighted composite (§3.3),

$$\mathrm{EachOperatorScore} = [w_c \cdot \delta_{\mathrm{pass}} + \sum_{i \in \mathrm{cases}} \frac{\delta_{\mathrm{accuracy},i}(w_f + w_p \cdot \mathrm{score}_i)}{\mathrm{num\_of\_cases}}] \cdot 100,$$

Benchmark	Golden Provenance	Scoring / Performance Metric	Test-case Generalization	Anti-Reward-Hacking	Maintenance
MultiKernelBench [10] (Huawei NPU tasks)	Author-provided reference	Compile / Correctness / Speedup	—	—	Academic
NPUKernelBench [11]	Author-provided reference	Compile / Correctness / Performance, reported as Pass@k	—	—	Academic (paired w/ AscendKernel-Gen)
AscendOptimizer bench [13]	`cann-ops` operators	Speedup only (optimization stage)	—	—	Academic
CANN Bench (ours)	**Vendor-authoritative (official CANN repo)**	**Three-dimensional composite score (see the Evaluation section)**	**20 public + 80 hidden per op**	**Concurrency / state-caching / environment defenses**	**CANN repo, versioned; online leaderboard (forthcoming)**

Table 1: Ascend ecosystem kernel-generation benchmarks: positioning of CANN Bench against relevant benchmarks targeting Ascend NPUs. Benchmarks targeting other ISAs (KernelBench, TritonBench, SOL-ExecBench, KernelCraft, NPUEval) are omitted from direct comparison. Columns report provenance of the golden reference, performance-score composition, test-case generalization discipline, anti-reward-hacking stance, and maintenance mode; "—" denotes an attribute that is not documented for a given benchmark.

where $\delta_{\text{pass}} \in \{0, 1\}$ is the operator-level compilation flag (a kernel that fails to compile zeros out every term), $\delta_{\text{accuracy},i} \in \{0, 1\}$ is the per-case functional indicator, score_i is the per-case performance sub-score, and w_c, w_f, w_p are linear weights (default $w_c = 0.2$, $w_f = 0.3$, $w_p = 0.5$). The performance sub-score score_i grades the candidate kernel's measured runtime $T_{\text{cand},i}$ between a PyTorch-on-Ascend baseline $T_{\text{baseline},i}$ and an analytically derived per-case hardware limit $T_{\text{HW},i}$ (§3.2, §3.3), following the fractional form of SOL-ExecBench [20] with anchors adapted to Ascend.

Precision scope. The current release covers FP16, BF16, FP32, and quantized integer paths (notably INT8 for weight and activation quantization). FP8, HiF8, MXFP8, FP4, and MXFP4 remain planned extensions discussed in §1.6.

Online leaderboard. CANN Bench is delivered with an online leaderboard[1].

1.5 Engineering Substrate and Anti-Reward-Hacking

Reliable scoring depends on the harness. CANN Bench runs compilation, accuracy verification, and performance sampling in a single pipeline. To control measurement noise on the NPU path, it applies frequency locking, L2-cache eviction between timed runs, and kernel-level profiling with msprof [26]. To isolate cross-case state, it executes each operator in a fresh subprocess. Performance baselines are versioned rather than fixed: `update_baseline_perf.py` refreshes `baseline_perf_us` as hardware and CANN versions change, so speedups are measured against the current attainable reference rather than a stale snapshot.

Our reward-hacking defense follows the three exploit classes detailed in §3.4: concurrency, state caching, and environment manipulation. The current release reduces these risks through constrained execution and repeated validation rather than complete closure. Submissions run on the default profiled stream; evaluation

[1]The portal is being prepared and will be released in a future update.

boundaries are synchronized; correctness is rechecked on regenerated inputs; and tensor storage is rotated through a clone pool to defeat simple cached-output reuse. Numerical validation uses dtype-aware thresholds, and leaderboard scores aggregate over both splits, so a submission cannot be tuned to disclosed inputs alone.

Two reward-hacking surfaces remain partially open: broader multi-stream execution is restricted rather than supported, and opaque pre-compiled binary loading remains an open review item. We treat the current design as a practical defense baseline; the deployed mitigations and open items are detailed in §3.4.

1.6 Status, Roadmap, and Open-Source Plan

As of this release, CANN Bench provides specifications, golden implementations, public test cases, and performance baselines for 53 operators across L1–L4. The harness and baseline-collection pipeline have completed end-to-end runs on NPU hardware. The public leaderboard is being prepared and will be released in a future update.

Three extensions are planned. The leaderboard portal will index and date every submission so that results stay directly citable. Kernel-DSL coverage will extend the initial Ascend-optimized support to Triton, PyPTO, TileLang, and other CANN-compatible paradigms under the same specification and scoring protocol. Precision coverage will expand the current FP16 / BF16 / FP32 / INT8 baseline to FP8, MXFP8, FP4, MXFP4, HiF8, and related low-precision formats as operators and baselines are validated.

The operator set, golden implementations, harness, and documentation are open-source. The project accepts contributions of new operators, test cases, baseline updates, and sub-leaderboards, so the benchmark tracks Ascend hardware and workload evolution over time.

Availability and license. CANN Bench is hosted in the official CANN repository at `https://gitcode.com/cann/cann-bench` and released under the *CANN Open Software License Agreement Version 2.0* [27]. The harness targets Ascend 910B2 and is validated against CANN $\geq$ 9.0.0, the version we recommend for reproducing the reported baselines; older toolkit releases are not guaranteed to match the published `baseline_perf_us` values, which are versioned alongside the harness via `inner/update_baseline_perf.py`.

2 Benchmark and Dataset

2.1 Design Principles

CANN Bench is organized around three principles for Ascend C kernel evaluation.

1. **Vendor-authoritative provenance.** Every workload in CANN Bench draws on official artifacts from the CANN repository, including standardized operator definitions, reference implementations, and standard test cases. CANN Bench is distinct from the vendor-maintained operator base `opbase`[2]: `opbase` is the production operator collection that ships with CANN, while CANN Bench is an evaluation benchmark for AI-generated kernels whose task set partially overlaps with `opbase` but is not identical. Even on shared tasks, CANN Bench specifications are not transcribed verbatim: we relax a subset of the original functional requirements—for example, narrowing the supported dtype and shape envelope or trimming peripheral code paths—so the evaluation focuses on the core kernel logic rather than the surrounding production machinery. Beyond these canonical operators, the benchmark also includes emerging custom operators absent from the official codebase, such as `Engram` from DeepSeek [28]. Combining canonical and newly proposed operators broadens the scope on which generation and optimization can be evaluated.
2. **Four levels with increasing difficulty.** Operators are grouped into four hierarchical levels (L1–L4), from element-wise kernels at L1 through progressively composite, contraction-heavy, and fused production kernels at L4. The levels also map to a hardware-utilization progression: L1 and L2 use only vector operations, L3 introduces basic combinations of vector and cube operations, and L4

[2] `https://gitcode.com/cann/opbase`.

Level	Count	Description	Type	Representative Operators
L1	8	Single-primitive kernels involving only Vector Core operations	Vector	`gelu`, `swi_glu`, `sigmoid`, `mish`
L2	16	Fused composite kernels involving complex Vector Core operations without matrix contractions	Vector	`rms_norm`, `cross_entropy_loss`, `softmax`, `apply_rotary_pos_emb`
L3	21	Kernels combining matrix contractions with Vector Core operations	Vector & Cube	`conv2_d`, `grouped_matmul`, `top_k`, `dequant_swiglu_quant`
L4	8	Complex fused kernels requiring coordinated Matrix and Vector Core pipelines within a single kernel	Vector & Cube	`sparse_flash_attention`, `mla_prolog`, `gru`, `lstm`
Total	**53**			

Table 2: A summary of CANN Bench. The four levels track a progression in implementation difficulty, from local pointwise kernels to production-grade fused pipelines.

requires tightly coordinated vector–cube pipelines within a single fused kernel. Table 2 summarizes the four levels. Our levels reflect how each operator maps onto Ascend's execution model, not shallow features such as code length or operator arity that prior kernel-generation benchmarks have used as difficulty proxies. The level at which an agent first stalls thus localizes its capability gap: a system that clears L1 and L2 cleanly but degrades sharply at L3 reveals a specific weakness on cube-side reasoning rather than producing only a lower aggregate score. The resulting per-level profile delivers the capability-tier separation that C1 calls for, not a single-axis difficulty score.

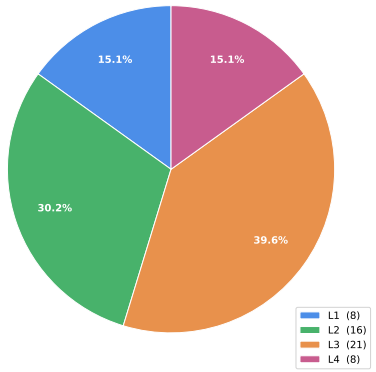
3. **Case diversity rather than single-shape scoring.** Each operator carries a family of concrete cases that vary tensor shape, dtype, and attribute settings. The design avoids rewarding kernels that overfit a single canonical configuration and instead tests whether generated code generalizes across the combinations that matter in real Ascend C development. Across the current split, the cases cover float16 / bfloat16 / float32 plus integer paths (int8, uint8, int32, int64) and mixed-dtype configurations.

2.2 Benchmark Structure and Dataset Overview

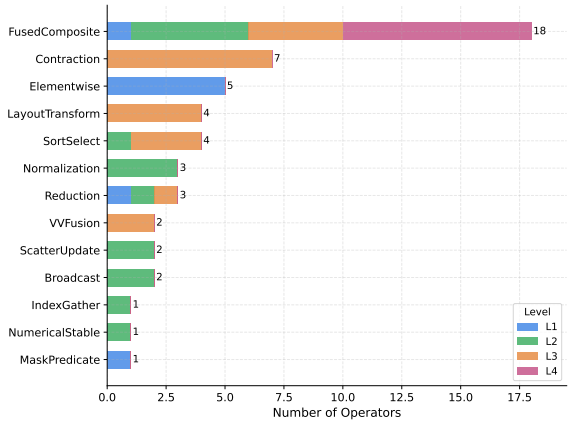
Table 2 summarizes the current CANN Bench release: **53 operators**, each with about 20 test cases. The benchmark is versioned alongside the CANN repository and grows as additional operators are promoted from the experimental track into the curated set. Figure 1 complements Table 2 with four views of the 1060 test cases.

Level distribution (Figure 1a). Operators are distributed across four hierarchical levels. The third level contains the most operators (21), spanning contraction, sorting, layout conversion, and fused quantization kernels under a heterogeneous mix of vector and cube workloads. The second level follows (16), highlighting the widespread use of multi-input normalization, indexing, and fused utility kernels in practical Ascend workloads. The first and fourth levels are tied for the fewest (8 each): L1 concentrates on single-primitive vector kernels, while L4 gathers the most engineering-heavy fused pipelines under the current classification.

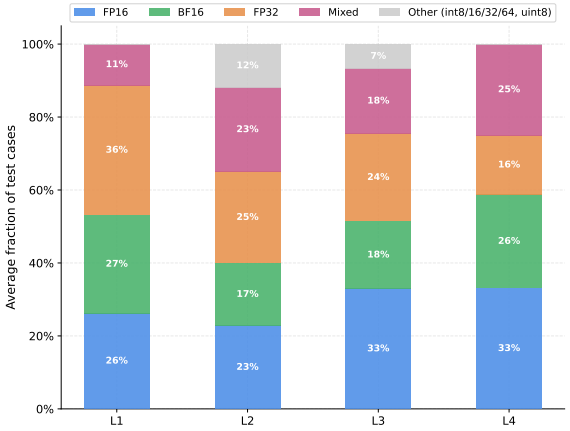
Operation category distribution (Figure 1b). The category labels span a wide range of operator types. *FusedComposite* is the largest category, followed by *Contraction*, *Elementwise*, and *SortSelect*. Composition differs across levels: L1 centers on elementwise kernels, L2 on fused composite and normalization operators, L3 on contraction and reordering-heavy kernels, and L4 consists entirely of fused composite operators. The distribution shows that CANN Bench extends beyond pure GEMM evaluation to indexing, reduction,



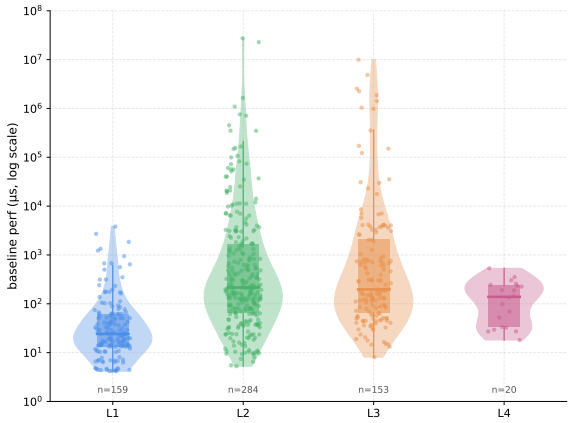
(a) Operator count per difficulty level.



(b) Operation category distribution by level.



(c) Primary compute dtype distribution per level.



(d) Baseline NPU execution time per level (μs, log scale).

Figure 1: Dataset overview of our CANN Bench. (a) Operator count per difficulty level. (b) Operation category distribution, stacked by level contribution. (c) Distribution of the primary compute data type per level. (d) Distribution of the provided baseline execution times (log-scale).

layout manipulation, and control-intensive operators—common failure points for AI-assisted Ascend C code generation.

Data-type distribution (Figure 1c). The figure reports the primary compute dtype of each operator. L1 and L2 are dominated by float16, bfloat16 and float32 paths, which matches the typical workload of activation, normalization and utility kernels. L3 and L4 contain a larger share of mixed-dtype and integer cases, as several operators in those tiers fuse floating-point activations with quantized weights, integer metadata, and multi-input specifications.

Performance baseline distribution (Figure 1d). Each test case in our dataset is associated with a baseline execution duration of the PyTorch reference implementation (in μs). Baselines in L1 cluster tightly at low values, while those in L2 and L3 exhibit far wider ranges, including high-duration outliers. L4 cases occupy a narrow mid-to-low duration band.

2.3 Test-Case Design Methodology

Each operator in CANN Bench carries a structured case set rather than a hand-picked collection of inputs. The construction follows four explicit coverage axes and ends with per-operator uniqueness constraints, so each case contributes evidence the others do not.

Category	Example range	Probes
Symmetric small	$[-1, 1]$, $[-0.1, 0.1]$	Typical activation regime
Symmetric medium	$[-10, 10]$, $[-100, 100]$	Wider numerical range
Symmetric large	$[-1000, 1000]$	Saturation, accumulator pressure
Asymmetric	$[-1, 2]$, $[-5, 10]$	Skewed distributions
Type boundary	fp16 $[-65504, 65504]$, fp32-exp $[-88, 88]$	Overflow / underflow
Special values	$\pm\infty$, NaN	Numerical stability
Zero	$[0, 0]$	Degenerate input path

Table 3: Value-range categories that populate test-case inputs. Each operator's 20-case public split is required to draw from at least the symmetric-small, type-boundary, and special-value groups, so that no operator is evaluated only on benign data.

Tensor-size coverage. Each case is sized by its total *input element count*—the sum of $\prod_i d_i$ over all of its input tensors—which spans roughly 10^6 to 3×10^8. The benchmark covers this range across three regimes: small cases (a few million elements) sit inside the on-chip buffers and surface tiling and launch-overhead issues, large cases (around 10^8 and above) force off-chip memory traffic and expose bandwidth-bound behavior, and a medium band (tens of millions) covers the bulk of production shapes. A kernel that overfits to a single regime cannot collect the per-case credit at the other two.

Axis alignment. We label a shape *aligned* when every dimension is a power of two (e.g. 1024, 2048, 4096, 8192), and *non-aligned* when at least one dimension is not. The proxy is grounded in the Ascend DMA contract, which requires source and destination addresses to be 32-byte aligned and transfer lengths to be multiples of 32 bytes; for every dtype used in the benchmark, a power-of-two innermost dimension automatically satisfies that contract, while a prime or odd dimension forces the kernel to handle residual tiles, masked tail blocks, and unaligned DMA. Within each operator's case set we require *at least 50% non-aligned cases*, where the ratio is computed over the operator's 20 public cases and a case counts as non-aligned whenever any of its input tensors has a non-power-of-two dimension. Inside the non-aligned subset we further bias toward primes (e.g. 1009, 1021, 1537, 4097, $1{,}000{,}003$) so that residual sizes do not collapse onto convenient divisors that an aggressive auto-tiler might still find friendly. This is the operational definition behind the alignment-stress requirement referenced in §1.5.

Dimensionality. The case set spans 1D through 5D inputs rather than freezing each operator at a single rank. Mixing ranks stresses tiling along axes other than the canonical innermost one and surfaces broadcast and reshape paths that a 2D-only or 3D-only test would not.

Value-range sweep. Inputs are drawn from a fixed taxonomy (Table 3) that covers symmetric and asymmetric ranges of three magnitudes, dtype boundaries (e.g. ±65504 for fp16, ±88 for the fp32 exponent), the special values $\pm\infty$ and NaN, and the all-zero degenerate case. For ranges containing $\pm\infty$, the data generator injects the corresponding extremes into the head and tail of the flattened tensor while filling the interior with regular random values, so that overflow handling, denormal flushing, and special-value propagation are observable side-by-side with normal arithmetic.

Uniqueness constraints. Within each operator's 20-case public split we require three properties to be unique across cases: total input element count, the JSON-serialized attribute combination, and the value range. The constraint blocks a common failure mode in hand-curated benchmarks where two cases differ only cosmetically and effectively count once. A self-check loop with at least three rounds of validation runs in the contributing pipeline before a new case set lands in the repository.

Public and hidden splits. The 20 public cases per operator support local iteration and inspection, while an additional 80 hidden cases per operator are retained inside the CANN Bench harness for leaderboard

evaluation. Hidden cases follow the same coverage spec and uniqueness constraints, so a kernel tuned only against the public split has no informational shortcut to the hidden one.

2.4 Problem Specification Format

Each task ships with four artifacts; SOL-ExecBench [20] adopts a similar but smaller set.

1. **Operator specification (`desc.md`).** A structured description covering the mathematical definition, interface contract (input/output shapes, dtypes, and constraints), and numerical accuracy tolerance.
2. **Operator prototype (`proto.yaml`).** A machine-readable YAML file capturing the operator name, category, schema, attributes, shape support, and supported data types for inputs and outputs.
3. **Test cases (`cases.csv`).** A CSV table whose rows are concrete test cases, each specifying input shapes, data types, hyper-parameters, and the baseline NPU execution time in microseconds.
4. **Golden implementation (`golden.py`).** A Python reference implementation, typically in PyTorch, that produces the expected output for each test case.

Together, the four difficulty levels, the case-design methodology, and the four-artifact specification format fix the dataset surface that any submission is evaluated against: which operators are in scope, which configurations are tested per operator, and what each task hands the candidate kernel as input. The next section turns from the dataset to the scoring side: how the harness converts a submission's behavior on these artifacts into compilation, functional, and performance signals, and how those signals are combined into a per-operator and benchmark-wide composite score.

3 Evaluation

CANN Bench scores each test case on three axes: compilation, functional correctness, and performance. The first two are binary gates; performance is continuous, anchored to a hardware-derived bound rather than a moving framework baseline. Linear weights combine the three into a per-operator composite.

For each test case, the harness records three reference times: T_{baseline}, the measured runtime of the operator's CANN-internal implementation on the target Ascend NPU; T_{HW}, an analytical hardware limit serving as a per-case lower bound; and T_{cand}, the measured runtime of the candidate kernel. The first two are computed once per case at benchmark-construction time; T_{cand} is collected at evaluation alongside the compilation and accuracy flags. Section 3.1 specifies the functional-correctness standard. Section 3.2 motivates the hardware-anchored design and defines T_{HW}. Section 3.3 gives the per-case performance score and assembles the three axes into per-operator and suite scores. Section 3.4 closes with the reward-hacking taxonomy and the mitigations the current evaluator deploys.

3.1 Functional-Correctness Standard

CANN Bench evaluates a kernel's output against a high-precision reference and gates each test case with a per-tensor relative-error rule. The current rule is the *double-thousandth / double-myriad* standard —summarized in Table 4—and the harness layers four engineering safeguards around it. The rule is intentionally simple at this stage; future iterations will tighten it as the operator base matures (see end of this subsection).

Per-element relative error. For each element i in an output tensor, let $\hat{y}_i$ denote the submitted kernel output and y_i the reference. The elementwise relative error is

$$e_i \;=\; \frac{|\hat{y}_i - y_i|}{|y_i| + 10^{-7}}. \tag{1}$$

The 10^{-7} guard avoids a degenerate division when y_i is exactly zero.

Dtype	Per-element bound τ_d	Outlier-ratio cap	Mnemonic
FP16	1×10^{-3}	1×10^{-3}	per-mille
BF16 (relaxed)	4×10^{-3}	4×10^{-3}	4-per-mille
FP32	1×10^{-4}	1×10^{-4}	per-myriad
Integer dtypes (notably INT8 for weight and activation quantization): exact equality is required.			
Lower-precision and emerging formats (FP8, HiF8, MXFP8, FP4, MXFP4): under standardization (§1.6).			

Table 4: Per-dtype precision thresholds. Each row uses one number τ_d to bound both the per-element relative error and the share of elements that may exceed it—hence the "double-thousandth / double-myriad" naming. BF16 is intentionally looser than FP16 because its narrower mantissa accumulates more per-element rounding error.

Outlier-ratio pass rule. For dtype d with threshold τ_d, an element is called an *outlier* when $e_i > \tau_d$. A tensor passes the accuracy check when the share of outliers is itself bounded by τ_d:

$$\frac{\left|\{\, i : e_i > \tau_d \,\}\right|}{N} \;\leq\; \tau_d, \tag{2}$$

where N counts the positions i at which both $\hat{y}_i$ and y_i are finite; positions containing $\pm\infty$ or NaN on either side are handled separately by the special-value rule (see below). The same number τ_d thus plays a dual role—it bounds the per-element error on one axis and bounds the share of points that may exceed that bound on the other—which is the origin of the "double-thousandth / double-myriad" naming. For multi-output operators, every output tensor must clear this gate independently.

Per-dtype thresholds. Table 4 lists τ_d for the main floating-point dtypes used in the benchmark. FP16 sits at one part per mille, FP32 tightens to one part per myriad, and BF16 is relaxed to four parts per mille: BF16 carries only 7 mantissa bits to FP16's 10, so the same arithmetic chain accumulates appreciably more rounding error per element, and a uniform FP16 threshold would over-penalize correct BF16 implementations. Integer dtypes are evaluated under exact equality. Lower-precision floating-point formats remain on the precision-standard roadmap and are not yet gated under this rule.

The four engineering safeguards below operate orthogonally to the threshold rule and apply to every dtype.

High-precision reference. The Golden function runs on CPU in FP64 for all floating-point inputs, which keeps the reference well above the device's native precision and prevents overflow or underflow during the reference computation itself from corrupting the truth. Before e_i is computed, the FP64 Golden is reduced to the kernel output's dtype (`golden = golden.to(output.dtype)`), so that the threshold τ_d is interpreted at the device dtype rather than at the reference dtype. Integer and boolean inputs retain their original dtype throughout, which keeps the path compatible with operators that reject a Double substitute (e.g. `Gcd` on integer inputs, `CrossEntropyLoss` on `int64` targets).

Special-value handling. $\pm\infty$ and NaN are excluded from the outlier statistics in Eq. (2), but their positions must match between the kernel output and the Golden, and $\pm\infty$ entries must additionally agree in sign. A divergence in special-value placement is reported as a hard failure rather than as a numerical disagreement, so that a kernel cannot bypass the gate by saturating boundary regions.

Two-trial fresh-input verification (opt-in). The harness ships an opt-in second-pass mode (`AccuracyEvaluator.evaluate_with_retry`) for the accuracy gate. When the second pass is requested, after a case clears the gate on the first trial the evaluator re-samples a fresh input batch through the same case generator—adding 0.01 to the first floating-point tensor to defeat seed reuse—and re-evaluates the same gate; only cases that clear both trials count toward functional correctness. A submission that caches the first-trial output by data pointer or toggles a `computed-once` flag fails the second trial with stale or garbage results. The default leaderboard pipeline currently invokes the single-trial path; the second-pass mode will be turned on by default in a subsequent release.

Strict tensor identity. The harness validates the kernel return value with `type(out) is torch.Tensor` rather than `isinstance`, which rejects `FakeTensor` and other lazy-evaluation wrappers that can satisfy an `isinstance` check without performing any real computation.

Roadmap toward a more scientific standard. The double-thousandth / double-myriad rule is deliberately simple and robust at the current stage of CANN Bench. The published design lays out two follow-on directions for the precision standard: (i) migrating from the ecosystem-operator standard currently in use to Huawei's commercial operator precision standard[3] as it stabilizes, and (ii) extending the per-dtype tier to cover the emerging low-precision formats already on the precision roadmap in §1.6 (FP8, HiF8, MXFP8, FP4, MXFP4). These refinements are co-developed with the CANN operator-base maintainers.

3.2 Performance anchor: the hardware limit

The most natural design for a kernel-generation benchmark is to score a submission by its speedup over a software reference. For a benchmark that aims to remain stable across years, this design has a fundamental defect: the reference drifts with framework and driver versions. A kernel that scored $2\times$ over last quarter's CANN reference may score only $1.4\times$ against a faster successor without any change to the kernel itself. Longitudinal comparison loses meaning, and a published number becomes a relative quantity that ages out within months.

SOL-ExecBench [20] addresses this on NVIDIA Blackwell by anchoring on a hardware-derived lower bound—a physical constant invariant under software-stack version. We adopt the same principle on Ascend, introducing a single analytical anchor—the hardware limit T_{HW}—as the upper end of the score, while retaining T_{baseline} as the lower anchor so that progress remains framed against the engineering reference users actually have access to. The framework-dependent component appears only at the lower end of the score; the upper end is anchored to the chip itself.

To make "how fast can a kernel run" precise on Ascend 910B2, two facts must be acknowledged: 910B2 is a heterogeneous architecture in which six execution units operate concurrently in time, and achievable performance depends on a non-trivial space of implementation choices.

Six-unit decomposition. Let

$$K \;=\; \{\,\text{cube},\ \text{vector},\ \text{mte1},\ \text{mte2},\ \text{mte3},\ \text{fixp}\,\}$$

denote the set of execution units that operate concurrently within a single AIC group. Cube performs matrix contractions in the L0A×L0B→L0C accumulator; Vector handles elementwise, reduction, and complex-activation work in UB; MTE2 and MTE3 are the GM↔UB data-movement channels; MTE1 issues L1→L0A/L0B prefetch for Cube; FixP is the L0C→GM write-back path that fuses simple epilogues such as bias, scale, quant, clip, and ReLU. The 24 AIC groups on a 910B2 die replicate this layout in parallel.

For a fixed kernel, let $t_k(\delta)$ denote the work time on unit k under implementation choice δ (work divided by peak rate), where δ ranges over active-core count, tile shape, schedule, buffer allocation, and pipeline depth. Because the six units overlap in time, the device-side dataflow time is bounded below by their maximum; the hardware limit is the minimization of this maximum across the implementation space:

$$T_{\text{HW}} \;=\; \min_{\delta}\ \max_{k\in K}\ t_k(\delta). \tag{3}$$

T_{HW} is the runtime a kernel would achieve under three idealizations: every unit operating at peak rate, perfect overlap across all six units, and zero host-side launch overhead. It depends only on the operator's intrinsic FLOP and byte counts and on the published 910B2 peak parameters; it remains constant across software-stack upgrades and CANN releases. No real implementation can violate this bound.

Derivation procedure. T_{HW} is computed once per case through an analytical procedure grounded in 910B2 architectural documentation and published peak parameters. The procedure takes as input the operator's

[3] `https://gitcode.com/cann/opbase/blob/master/docs/zh/ops_precision_standard/commercial_standard.md`.

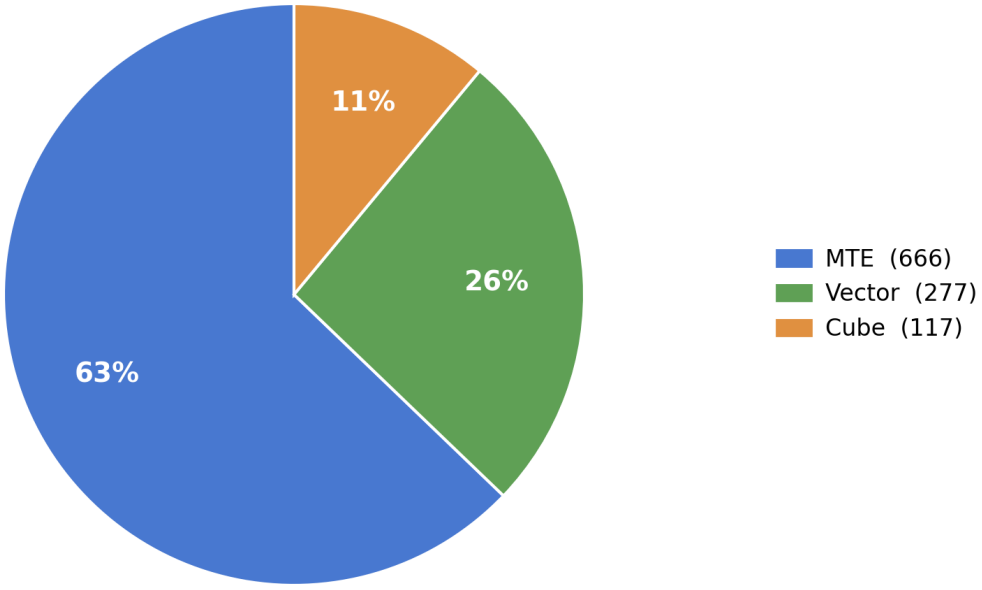


Figure 2: Distribution of dominant hardware bottlenecks across the 1060 cases in CANN Bench.

mathematical definition, its interface contract, and the case's shape and dtype. For each unit $k \in K$ it estimates t_k from the operator's FLOP and byte counts and that unit's peak rate, takes the per-unit maximum, and minimizes over a structured implementation-choice space δ that respects on-chip buffer constraints (L1 capacity, L0A/L0B capacity, L0C accumulator size). The procedure outputs T_{HW} together with the dominant-bottleneck label. The value of T_{HW} is determined entirely by the hardware specification: should the target hardware platform or its six-unit characterization be revised, T_{HW} is regenerated accordingly.

Per-case anchor pair. Each case is characterized by the analytical anchor pair $(T_{\text{baseline}}, T_{\text{HW}})$, which combines at evaluation time with the candidate kernel's measured runtime T_{cand}. By the physical meaning of the hardware limit, $T_{\text{HW}} \leq T_{\text{baseline}}$ should hold on every case. Any observation that violates this inequality triggers an evaluation review as an audit signal on either the analytical lower-bound derivation or the timing measurement.

Bottleneck Attribution Each test case is annotated with the dominant hardware bottleneck that limits T_{HW}, classified into three categories: *Vector* (Vector Core arithmetic), *Cube* (Cube Core matrix computation), and *MTE* (Memory Transfer Engine for data movement, rolling up MTE1/MTE2/MTE3 and the FixP writeback path). As shown in Figure 2, MTE dominates at 63% of the 1060 cases, with Vector at 26% and Cube at 11%. This distribution reflects the full Ascend pipeline workload, not just the compute engines.

3.3 Per-case performance score and composite scoring

Per-case performance score Let T_{cand} denote the candidate kernel's device-side measured runtime. We adopt the fractional form of SOL-ExecBench [20], rewritten with the Ascend hardware-limit anchor:

$$\text{Score}_i = \frac{T_{\text{baseline},i} - T_{\text{HW},i}}{(T_{\text{cand},i} - T_{\text{HW},i}) + (T_{\text{baseline},i} - T_{\text{HW},i})}. \tag{4}$$

The score has three anchor properties:

- $T_{\text{cand}} = T_{\text{baseline}} \Rightarrow \text{Score} = 0.5$ (matching the reference implementation is the midpoint);

- $T_{\text{cand}} = T_{\text{HW}} \Rightarrow \text{Score} = 1$ (matching the hardware limit is a perfect score);
- $T_{\text{cand}} \to \infty \Rightarrow \text{Score} \to 0$ (slower kernels decay smoothly toward zero).

The midpoint design separates three regimes on a single bounded scale: worse than the reference implementation ($\text{Score} < 0.5$); better than the reference implementation but sub-limit ($0.5 < \text{Score} < 1$); and at or beyond the hardware limit ($\text{Score} \geq 1$). Submissions that exceed the limit are not capped at 1; the formula remains well-defined and grows past 1, but the case is flagged for review, since $T_{\text{cand}} < T_{\text{HW}}$ implies one of three things: an under-tight T_{HW} derivation, a measurement-side exploit (§3.4), or a genuine modelling gap in the analytical procedure. Each merits case-by-case investigation rather than silent absorption into the score.

Compared with a raw baseline ratio $T_{\text{baseline}}/T_{\text{cand}}$, equation (4) is bounded, comparable across operators, and remains well-defined when T_{baseline} is already close to the hardware limit. The formula awards similar credit for similar fractional progress within the reference-to-limit window, regardless of how generous the reference happens to be on the case at hand.

Composite Score Equation (4) grades a single test case along the performance axis. A submission, however, is ranked at the operator level, and a leaderboard entry covers all evaluated operators. We therefore lift the per-case score to a per-operator composite that also folds in compilation and functional correctness.

Three signals enter the composite. At the operator level, a single compilation flag $\delta_{\text{pass}} \in \{0, 1\}$ records whether the submitted source compiles; under the official "one submission per operator" protocol this flag applies to the entire operator rather than to individual cases. At the case level, each case carries a functional indicator $\delta_{\text{accuracy},i} \in \{0, 1\}$ (does the kernel match the golden reference under the per-dtype precision standard) and a per-case performance score score_i from Equation (4). The three axes are combined under linear weights $w_c = 0.2$, $w_f = 0.3$, $w_p = 0.5$ for compilation, functional correctness, and performance, with $\delta_{\text{accuracy},i} \equiv 0$ whenever $\delta_{\text{pass}} = 0$ (a kernel that fails to compile cannot pass any case). The per-operator composite averages the weighted contributions across the operator's cases and is rescaled to a 0–100 range:

$$\text{EachOperatorScore} = \left[w_c \cdot \delta_{\text{pass}} + \sum_{i \in \text{cases}} \frac{\delta_{\text{accuracy},i} \, (w_f + w_p \cdot \text{score}_i)}{\text{len}(\text{cases})} \right] \cdot 100, \tag{5}$$

where the operator-level flag δ_{pass} is replicated across every case in the sum, so the compilation term contributes a constant $w_c \cdot \delta_{\text{pass}}$ to the average regardless of how many cases the operator carries. The structure is therefore additive rather than multiplicative: compilation contributes a single operator-wide w_c floor on success, and each functionally passing case independently contributes a fixed w_f floor plus a performance term $w_p \cdot \text{score}_i$ that scales with the hardware-anchored fractional score of Equation (4), before the whole quantity is normalised by $\text{len}(\text{cases})$ and lifted onto a 0–100 scale. Per-level and benchmark-level totals are simple sums over the operators they cover:

$$\text{Score}_{\text{Level-}N} = \sum_{\text{op} \in \text{Level-}N} \text{EachOperatorScore}_{\text{op}}, \qquad \text{Score}_{\text{benchmark}} = \sum_{N=1}^{4} \text{Score}_{\text{Level-}N}. \tag{6}$$

This formulation keeps the three axes co-active. A submission that fails to compile contributes nothing on any axis, since both the compilation term and every case-level term collapse to zero. A submission that compiles but fails the functional gate on a given case forfeits both the 0.3 floor and the $0.5 \cdot \text{score}_i$ term on that case while preserving the 0.2 compilation contribution and the contributions from any cases that do pass. A functionally correct but slow submission keeps the 0.3 floor on every passing case and only loses (part of) the $0.5 \cdot \text{score}_i$ component. The weights $(w_c, w_f, w_p) = (0.2, 0.3, 0.5)$ are normalised to sum to 1, so a fully compiled, functionally correct, hardware-limit-matching operator scores exactly 100. The per-case ratio $w_p/w_f = 5/3$ keeps two extreme strategies on comparable footing: a fully correct submission at the baseline anchor ($\text{score}_i = 0.5$ on every case) scores 75 on the operator, while a partial-but-fully-optimised submission only overtakes that by clearing more than $11/16$ of its cases (roughly 14 of the current 20).

Category	Exploit Description	Defense Mechanism
Concurrency	**Thread injection:** Hide work in background Python threads.	Subprocess isolation; end-of-step synchronization.
	Stream injection: Launch work on unobserved non-default streams.	Planned all-stream synchronization before trace close.
	Graph-capture diversion: Reuse graph replay to avoid recomputation.	Second correctness trial with regenerated inputs.
State caching	**Cached-output reuse:** Return memoized outputs keyed by input identity.	Clone-pool address rotation.
	Lazy evaluation: Delay compute until validation.	Strict `torch.Tensor` type check.
	One-shot correctness: Compute once, then return placeholders.	Multiple correctness trials with perturbed inputs.
Environment	**Timing-API monkey-patching:** Override measurement functions.	Callable identity snapshot and restore.
	Precision downgrade: Run FP32 cases in FP16/BF16 then upcast.	Dtype-specific numerical thresholds.
	Pre-compiled binary embedding: Load opaque binaries at runtime.	Manual review only; open item.

Table 5: Observed or anticipated reward-hacking strategies against `CANN Bench` and corresponding defenses.

3.4 Reward Hacking and Mitigation

The composite score in Eq. (5) is only as honest as the measurements that feed it. Recent work on LLM-based kernel generation has shown that benchmark design matters as much as model capability: if the evaluator has loopholes, an optimizer may improve its score without improving the underlying kernel. In this literature, such behavior is commonly described as *reward hacking*, *benchmark exploitation*, or *evaluation loophole exploitation* [9, 20]. An NPU benchmark is exposed to the same reward-hacking strategies as a GPU benchmark, because both rely on asynchronous device execution, host-side timing logic, memory reuse, and framework-managed runtime state. We have already covered the functional-correctness side of this story in §3.1; this subsection turns to the performance side and the additional defenses that the harness layers around timing, state, and environment.

Among prior benchmark studies, SOL-ExecBench gives a clear taxonomy of these exploits for agentic kernel optimization. It groups the observed strategies into three families: *concurrency exploit*, *state caching*, and *environment manipulation* [20, §4.4.1, Table 3]. We adopt the same high-level grouping for `CANN Bench`, as the same classes of exploits remain plausible on Ascend (Table 5).

On Ascend, the relevant interfaces are exposed through the `torch.npu`/`torch_npu` stack: timing and synchronization use APIs such as `torch.npu.Event.elapsed_time` and `torch.npu.synchronize`; stream management uses `torch.npu.Stream`; profiling is available through `torch_npu.profiler.profile`; and graph-based execution support is also present in the public `torch_npu` codebase [29–31]. As a result, the same reward-hacking patterns identified for CUDA benchmarks remain relevant when evaluating Ascend custom operators.

The current evaluator makes two conservative design choices to reduce the remaining reward-hacking surface, even though they limit some functionality.

First, we discourage user-managed stream orchestration. Submitted kernels are expected to run on the default execution stream—the one observed by the profiler—and multi-stream submissions are currently not ranked. This choice simplifies enforcement because stream-level visibility is still relatively fragile across `torch_npu` versions [30, 31].

Second, we rely on the default framework allocator rather than enforcing our own global fresh-address policy. As a result, the clone pool is the main source of address diversity during timing. At current benchmark scales

this is sufficient, but it may become a limitation if future cases use much larger inputs. In the long term, this restriction could be relaxed if the allocator becomes controllable through an environment variable analogous to PyTorch's `PYTORCH_NO_CUDA_MEMORY_CACHING` [32].

Neither restriction should be viewed as permanent. Once the Ascend runtime exposes a stable mechanism for enumerating or synchronizing all streams across software versions, and once allocator behavior can be controlled more directly, `CANN Bench` can support a broader set of submissions without sacrificing evaluator robustness.

4 Conclusion

We introduced **CANN Bench**, an Ascend-oriented operator-generation benchmark designed to give AI-generated kernels under Huawei's CANN stack a shared, vendor-aligned evaluation scale. The initial release ships 53 operators and 1060 test cases distributed across four difficulty levels (L1–L4), each operator accompanied by a self-contained reproducibility kit—`desc.md`, `proto.yaml`, `cases.csv`, and `golden.py`—and scored on the three axes that matter for kernel work on Ascend 910B2: compilation, functional correctness, and performance. Performance is graded between a PyTorch-on-Ascend baseline and an analytically derived per-case hardware limit, so absolute scores carry a hardware-grounded meaning rather than only a relative ranking. The harness, baseline-collection pipeline, and operator suite are co-located in the official CANN repository and released under the CANN Open Software License v2.0, with an online leaderboard prepared as the durable maintenance surface.

Relative to existing kernel-generation benchmarks, CANN Bench occupies a position that none of the closest prior efforts simultaneously cover. Cross-platform suites such as KernelBench, TritonBench, and SOL-ExecBench target NVIDIA silicon and do not transfer mechanically to Ascend's tiling semantics and multi-level buffer structure; MultiKernelBench spans three hardware backends but treats Ascend as one slice with limited per-operator depth; and NPUKernelBench is co-evolved with a specific training recipe and includes a static-shape track where a single canonical input configuration governs an entire task. CANN Bench differs in three concrete ways: golden references are vendor-authoritative and maintained alongside official CANN operator contracts; every operator carries a 20-case public split for local iteration plus an 80-case hidden split for leaderboard evaluation, which prevents fitting to disclosed inputs; and the harness layers explicit defenses—subprocess isolation, end-of-step synchronization, clone-pool address rotation, dtype-aware numerical thresholds, and second correctness trials with regenerated inputs—against the concurrency, state-caching, and environment exploit classes that have already been observed against GPU benchmarks.

We are explicit about the current limits of this release. Precision coverage is restricted to FP16, BF16, FP32, and INT8; FP8, HiF8, MXFP8, FP4, and MXFP4 are deliberate near-term extensions rather than shipped capabilities. Two reward-hacking surfaces remain partially open: multi-stream execution is restricted rather than ranked, because stream-level visibility across `torch_npu` versions is still fragile; and opaque pre-compiled binary loading is presently addressed by manual review rather than automated detection. Address diversity during timing depends on the framework allocator's behavior through a clone pool, which is sufficient at current case scales but will need a controllable allocator path—analogous to PyTorch's `PYTORCH_NO_CUDA_MEMORY_CACHING`—as input sizes grow. The benchmark also begins on a single hardware target (Ascend 910B2) and a single kernel programming surface, so coverage of newer Ascend silicon and additional DSLs is a near-term task rather than an established result.

Looking forward, CANN Bench is intended to evolve with the Ascend ecosystem rather than freeze at release. Three extensions are scheduled: the online leaderboard portal will index and date every submission so that results stay directly citable; kernel-DSL coverage will extend beyond the initial Ascend-optimized path to Triton, PyPTO, TileLang, and other CANN-compatible paradigms under the same specification and scoring protocol; and precision coverage will expand to FP8, MXFP8, FP4, MXFP4, HiF8, and related low-precision formats as operators and baselines are validated. Versioned performance baselines, refreshed through `update_baseline_perf.py`, ensure that reported speedups continue to track the current attainable reference rather than a stale snapshot as CANN and Ascend hardware progress. The operator set, golden implementations, harness, and documentation are open-source, and the project actively accepts contributions

of new operators, test cases, baseline updates, and sub-leaderboards. By coupling vendor-authoritative golden references, a hardware-grounded performance anchor, a hidden-split evaluation protocol, and a deployed anti-reward-hacking harness inside the official CANN repository, CANN Bench aims to serve as the shared, sustained evaluation scale that AI-generated kernels on Ascend have so far lacked, and to grow alongside the workloads, precisions, and silicon it is built to measure.

References

[1] Anonymous. CUDA Agent: Large-scale agentic RL for high-performance CUDA kernel generation. *arXiv preprint arXiv:2602.24286*, 2026.

[2] Anonymous. CuTeGen: An LLM-based agentic framework for generation and optimization of high-performance GPU kernels using CuTe. *arXiv preprint arXiv:2604.01489*, 2026.

[3] Anonymous. AutoKernel: Autonomous GPU kernel optimization via iterative agent-driven search. *arXiv preprint arXiv:2603.21331*, 2026.

[4] Martin Andrews and Sam Witteveen. GPU kernel scientist: An LLM-driven framework for iterative kernel optimization. *arXiv preprint arXiv:2506.20807*, 2025.

[5] Jianghui Wang, Vinay Joshi, Saptarshi Majumder, Xu Chao, Bin Ding, Ziqiong Liu, Pratik Prabhanjan Brahma, Dong Li, Zicheng Liu, and Emad Barsoum. GEAK: Introducing triton kernel AI agent and evaluation benchmarks. *arXiv preprint arXiv:2507.23194*, 2025.

[6] Carlo Baronio, Pietro Marsella, Ben Pan, Simon Guo, and Silas Alberti. Kevin: Multi-turn RL for generating CUDA kernels. *arXiv preprint arXiv:2507.11948*, 2025.

[7] Anonymous. Dr. Kernel: Reinforcement learning done right for Triton kernel generations. *arXiv preprint arXiv:2602.05885*, 2026.

[8] Ping Guo, Chenyu Zhu, Siyuan Chen, Fei Liu, Xi Lin, Zhichao Lu, and Qingfu Zhang. EvoEngineer: Mastering automated CUDA kernel code evolution with large language models. *arXiv preprint arXiv:2510.03760*, 2025.

[9] Robert Tjarko Lange, Qi Sun, Aaditya Prasad, Maxence Faldor, Yujin Tang, and David Ha. Towards robust agentic cuda kernel benchmarking, verification, and optimization, 2025.

[10] Z. Wen, Y. Zhang, Z. Li, Z. Liu, L. Xie, and T. Zhang. MultiKernelBench: A multi-platform benchmark for kernel generation. *arXiv preprint arXiv:2507.17773*, 2025.

[11] Xinzi Cao, Jianyang Zhai, Pengfei Li, Zhiheng Hu, Cen Yan, Bingxu Mu, Guanghuan Fang, Bin She, Jiayu Li, Yihan Su, Dongyang Tao, Xiansong Huang, Fan Xu, Feidiao Yang, Yao Lu, Chang-Dong Wang, Yutong Lu, Weicheng Xue, Bin Zhou, and Yonghong Tian. AscendKernelGen: A systematic study of LLM-based kernel generation for neural processing units, 2026.

[12] Zhongzhen Wen, Shudi Shao, Zhong Li, Yu Ge, Tongtong Xu, Yuanyi Lin, and Tian Zhang. AscendCraft: Automatic Ascend NPU kernel generation via DSL-guided transcompilation. *arXiv preprint arXiv:2601.22760*, 2026.

[13] Jiehao Wu, Zixiao Huang, Wenhao Li, Chuyun Shen, Junjie Sheng, and Xiangfeng Wang. AscendOptimizer: Episodic agent for Ascend NPU operator optimization. *arXiv preprint arXiv:2603.23566*, 2026.

[14] Yujie Zheng, Zhuo Li, Shengtao Zhang, Hanjing Wang, Junjie Sheng, Jiaqian Wang, Junchi Yan, Weinan Zhang, Ying Wen, Bo Tang, and Muning Wen. Towards cold-start drafting and continual refining: A value-driven memory approach with application to NPU kernel synthesis. *arXiv preprint arXiv:2603.10846*, 2026.

[15] KernelCAT Team. KernelCAT: An expert-level agent for compute acceleration on Ascend NPU. Zhihu article, 2025. Industrial technical blog post.

[16] Bita Darvish Rouhani, Ritchie Zhao, Ankit More, Mathew Hall, Alireza Khodamoradi, Summer Deng, Dhruv Choudhary, Marius Cornea, Eric Dellinger, Kristof Denolf, et al. Microscaling data formats for deep learning. *arXiv preprint arXiv:2310.10537*, 2023.

[17] Albert Tseng, Tao Yu, and Youngsuk Park. Training LLMs with MXFP4. *arXiv preprint arXiv:2502.20586*, 2025.

[18] Anne Ouyang, Simon Guo, Simran Arora, Alex L. Zhang, William Hu, Christopher Ré, and Azalia Mirhoseini. KernelBench: Can LLMs write efficient GPU kernels? *arXiv preprint arXiv:2502.10517*, 2025.

[19] Jianling Li, Shangzhan Wang, Zhenye Zhang, et al. TritonBench: Benchmarking large language model capabilities for generating Triton operators. In *Findings of ACL*, 2025. arXiv:2502.14752.

[20] Edward Lin, Sahil Modi, Siva Kumar Sastry Hari, Qijing Huang, et al. SOL-ExecBench: Speed-of-light benchmarking for real-world GPU kernels against hardware limits. *arXiv preprint arXiv:2603.19173*, 2026.

[21] Sarunas Kalade and Graham Schelle. NPUEval: Optimizing NPU kernels with LLMs and open source compilers, 2025.

[22] Jiayi Nie, Haoran Wu, Yao Lai, Zeyu Cao, Cheng Zhang, Binglei Lou, Erwei Wang, Jianyi Cheng, Timothy M. Jones, Robert Mullins, Rika Antonova, and Yiren Zhao. Kernelcraft: Benchmarking for agentic close-to-metal kernel generation on emerging hardware, 2026.

[23] Scaling Intelligence Lab. KernelBench v0.1. Stanford Scaling Intelligence Lab blog post, blog page, 2025.

[24] DeepReinforce Team. Hacks and defenses in automatic GPU kernel generation. project page, 2025.

[25] Scaling Intelligence Lab. Fall 2025 KernelBench maintenance and improvement plan. GitHub Issue #74, issue page, 2025.

[26] Huawei Ascend. msprof profiling tool: Ascend CANN auxiliary development toolkit documentation. Huawei Ascend documentation page, 2023. CANN Commercial 6.0.1.

[27] Huawei. CANN open software license agreement version 2.0. CANN repository LICENSE, 2026.

[28] Xin Cheng, Wangding Zeng, Damai Dai, Qinyu Chen, Bingxuan Wang, Zhenda Xie, Kezhao Huang, Xingkai Yu, Zhewen Hao, Yukun Li, Han Zhang, Huishuai Zhang, Dongyan Zhao, and Wenfeng Liang. Conditional memory via scalable lookup: A new axis of sparsity for large language models, 2026.

[29] Huawei. Npu and cuda function alignment. Huawei Technical Support Documentation. Accessed 2026-04-23.

[30] ModelScope SWIFT Contributors. Ascend npu performance data collection. Megatron-SWIFT Documentation. Accessed 2026-04-23.

[31] Ascend. Ascend extension for pytorch. GitHub repository. Accessed 2026-04-23.

[32] PyTorch Contributors. Cuda environment variables. PyTorch Documentation, 2025. Accessed 2026-04-23.